

Bases de datos gestionadas

Bases de datos gestionadas

De ficheros a filas y relaciones

Hasta ahora tus datos han vivido en ficheros: un front en S3, unas imágenes en EFS. Hoy llega la primera pieza que necesita algo más estructurado.

Dos piezas nuevas

Base de datos relacional

Guarda la información en tablas con filas y columnas, relacionadas entre sí — un libro pertenece a una categoría, aparece en varios préstamos.

Motor de base de datos

El programa que la gestiona (como PostgreSQL): guarda los datos en disco, responde consultas y evita que se corrompan aunque lleguen a la vez.

Instalar frente a consumir como servicio

Instalar PostgreSQL en una EC2 te da control total. RDS —Relational Database Service— se queda con buena parte del trabajo operativo, a cambio de que renuncies al sistema operativo por debajo.

	Instalada por ti	Gestionada (RDS)
Tamaño de cómputo	Tipo de instancia EC2	Clase db.* — mismo catálogo, prefijo db.
Parches del motor	Los aplicas tú	AWS, en ventana de mantenimiento
Copias de seguridad	Las configuras y vigilas tú	Automáticas, con recuperación a un punto
Conmutación por error	La montas tú, si la necesitas	Incluida si activas Multi-AZ
Acceso al sistema operativo	Total	Ninguno — ni por SSH
Versión exacta y extensiones	Total	El que RDS permita para ese motor

El mismo trabajo, en manos distintas

Coche propio, o coche con conductor

Con el tuyo decides el mecánico, el taller, cuándo lo revisas — pero si se avería un domingo, el problema es tuyo. Con el servicio, no eliges el mecánico ni ves el motor por dentro, pero si falla, no es tu problema resolverlo: te mandan otro. Ninguna opción es «la buena» en abstracto — es la escalera IaaS→PaaS→SaaS de la sesión 1, aplicada a bases de datos.

RDS no es solo PostgreSQL

Hoy trabajas con PostgreSQL porque es el motor de la Actividad 3.2, pero RDS gestiona también MySQL, MariaDB, SQL Server, Oracle, e incluso Aurora — un motor propio de AWS compatible con MySQL y PostgreSQL pero reescrito para ir más rápido y tolerar fallos mejor.

Subred privada y grupos de seguridad de datos

La base de datos va, sin excepción, en subred privada. RDS necesita su propio grupo de seguridad: una única regla de entrada, el puerto de la base de datos, y solo desde el grupo de seguridad de la aplicación.

 Internet



 RDS
(subred privada)

 Instancia de
la aplicación

*puerto 5432
solo desde su SG →*

 RDS
(subred privada)

No es un escenario hipotético

Bases de datos con el puerto abierto a 0.0.0.0/0 y sin contraseña son de los hallazgos más habituales al escanear internet. «Solo desde el grupo de seguridad de la aplicación» no es una buena práctica opcional — es la diferencia entre privada de verdad y privada solo en apariencia.

La contraseña no se escribe en ningún sitio

Escribir la contraseña en el código o en un fichero que subes al repositorio significa que cualquiera con acceso a ese código la tiene también a ella. Secrets Manager la guarda cifrada y fuera del código.

 RDS genera
la contraseña



 Secrets Manager
(la guarda cifrada)

 Tu aplicación

*la pide en el arranque
se la entrega →*

 Secrets Manager

La misma idea que vas a generalizar en el Tema 5: ningún secreto escrito en el código ni en el repositorio, sea cual sea la credencial.

Cómo llega esto a tu aplicación, en la práctica

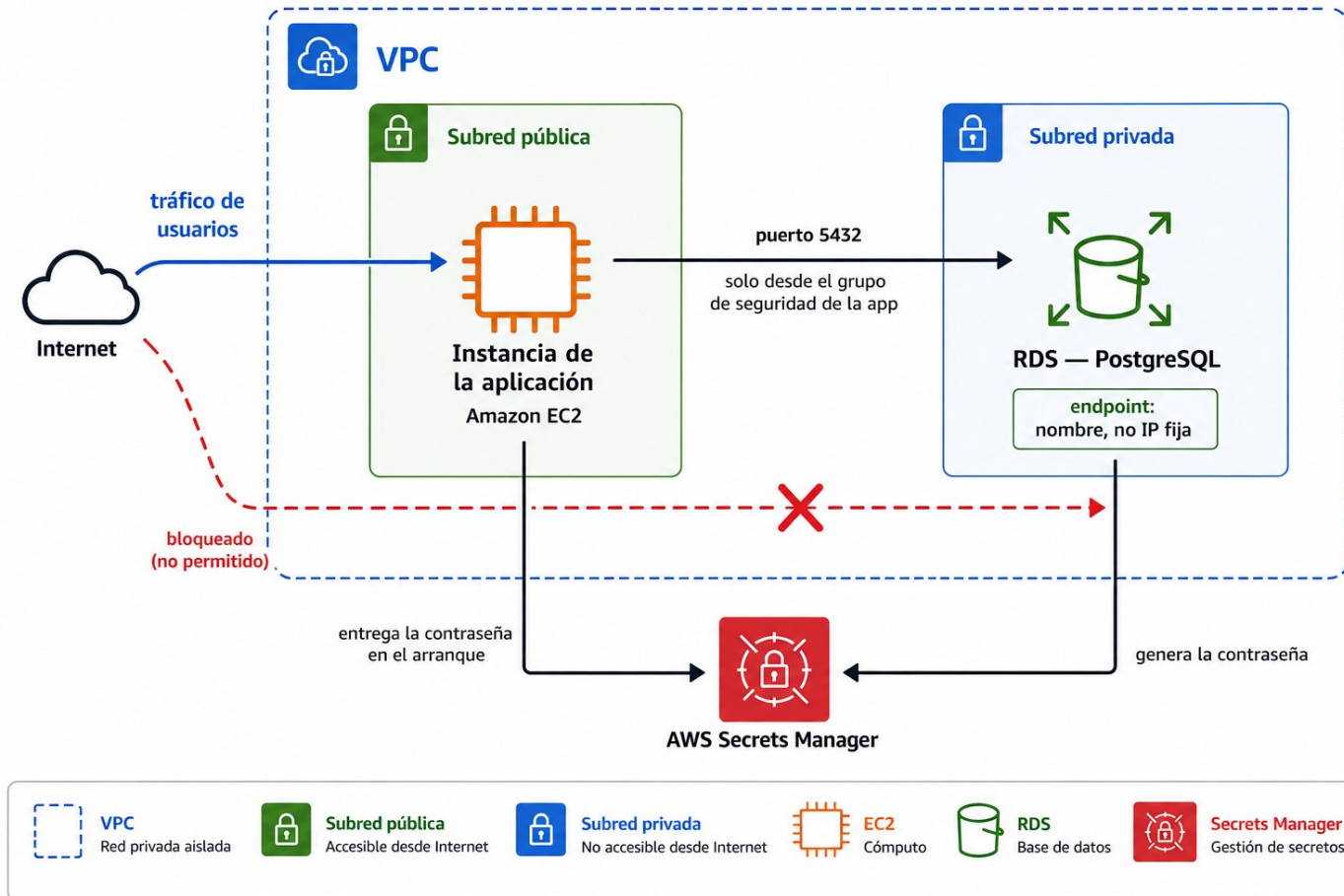
En el arranque de la instancia —el mismo user data de la Actividad 2.3— un script pide el endpoint y la contraseña por CLI, y los deja como variables de entorno, nunca escritos a mano:

```
# El endpoint de RDS: sustituye a la IP fija de siempre
export DB_HOST=$(aws rds describe-db-instances \
  --db-instance-identifier mi-base-de-datos \
  --query "DBInstances[0].Endpoint.Address" --output text)

# La contraseña, tal como la guardó Secrets Manager
aws secretsmanager get-secret-value \
  --secret-id mi-base-de-datos-secret \
  --query "SecretString" --output text
```

Con DB_HOST y DB_PASSWORD como variables de entorno, la aplicación se conecta igual que a cualquier base de datos: el código no sabe de dónde vienen esos valores. Eso hace posible migrar de una base de datos a otra sin tocar una línea del código.

Todas las piezas, en una sola foto



Subred privada, grupo de seguridad, endpoint y Secrets Manager encajando juntos.

Multi-AZ y réplicas de lectura

Dos mecanismos para repartir la base de datos entre zonas — resuelven problemas distintos

	Multi-AZ	Réplica de lectura
Para qué existe	Alta disponibilidad	Rendimiento de lectura
¿Se puede leer o escribir?	No, pasiva hasta el failover	Sí, solo lectura, en paralelo
Si falla la principal	AWS conmuta sola, en minutos	No conmuta sola — hay que promoverla

Multi-AZ no es gratis: duplica el coste de cómputo

La copia en espera no es una foto barata — es una instancia completa, del mismo tamaño que la principal, funcionando siempre aunque nunca respondas una consulta contra ella. Para una base de datos de desarrollo, es una garantía que no necesitas.

Backups automáticos frente a snapshots manuales

Los backups automáticos son cómodos —RDS los hace solo, cada día— pero tienen letra pequeña: si borras la instancia, desaparecen con ella.

	Backup automático	Snapshot manual
¿Quién lo crea?	RDS, solo, cada día	Tú, cuando lo pides
¿Cuánto dura?	La ventana de retención elegida	Hasta que tú lo borres
¿Sobrevive a borrar la instancia?	No	Sí
Para qué sirve	Recuperación a un punto reciente	Conservar un estado a largo plazo

Borrar la base de datos no borra tu snapshot manual

Es al revés de lo que se espera la primera vez: si borras una instancia que solo tiene backups automáticos, esos backups se pierden con ella. Un snapshot manual creado antes sigue disponible — por eso sirve como «punto de seguridad» antes de un cambio arriesgado.

Cambiar la clase de instancia después de crearla

La clase db.* que elegiste al crear la base de datos no es definitiva — es el mismo escalado vertical que ya conoces de EC2, aplicado a una base de datos, sin migrar los datos a mano.

Sin Multi-AZ

El motor se reinicia: la base de datos deja de responder unos segundos o minutos mientras se aplica el cambio.

Con Multi-AZ

RDS puede aplicar el cambio primero en la copia en espera y conmutar a ella, reduciendo la interrupción real que nota tu aplicación.

Relacional frente a NoSQL: el patrón de acceso

RDS no es la única familia de base de datos en la nube

Modelo	Servicio en AWS	Cuándo encaja
Relacional	RDS	Datos estructurados con relaciones claras — un catálogo con categorías y pedidos
Clave-valor	DynamoDB	Sesiones de usuario, carritos de compra temporales
Documental	DocumentDB	Catálogos con atributos muy distintos entre productos

La biblioteca, tres preguntas: préstamos activos con título y categoría cruza tablas (RDS); la sesión de un usuario navegando es un simple identificador (DynamoDB); una ficha con campos muy distintos según el tipo de libro encaja mejor sin columnas fijas (DocumentDB).

DynamoDB y DocumentDB, en detalle

DynamoDB

No tiene columnas fijas: cada elemento es un documento con los atributos que quieras. Solo decides de antemano la clave de partición (por ejemplo `sesion_id`). El acceso normal es siempre por esa clave — no un `SELECT ... WHERE` sobre cualquier campo. Sin motor que arrancar ni parches: se factura por lo que usas.

DocumentDB

Usa una arquitectura de clúster parecida a Aurora — no sirve nada sin al menos una instancia dentro. No se habla en SQL: es compatible con el protocolo de MongoDB, con comandos como `db.coleccion.find({...})` sobre documentos, sin obligar a que todos compartan el mismo esquema.

El criterio no es «cuál es más moderno», sino qué forma tienen tus datos y cómo los vas a consultar. En la Actividad 3.2 creas una tabla DynamoDB real; DocumentDB se queda en razonamiento.

Ideas clave

1

Instalar tu propia base de datos da control total; RDS asume parches, copias y conmutación por error a cambio del acceso al sistema operativo.

2

La base de datos va siempre en subred privada, con un grupo de seguridad que solo acepta tráfico desde la instancia de la aplicación.

3

Secrets Manager guarda la contraseña cifrada y fuera del código — puede generarla él mismo al crear la instancia.

4

La aplicación recibe endpoint y contraseña como variables de entorno que rellena un script de arranque por CLI.

5

Multi-AZ resuelve disponibilidad (copia pasiva que conmuta sola); una réplica de lectura resuelve rendimiento — y Multi-AZ duplica el coste.

6

La recuperación a un punto en el tiempo usa backups automáticos, que desaparecen si borras la instancia; un snapshot manual, no.

7

La clase db.* se puede cambiar después de crear la base de datos, a costa de una breve interrupción salvo que Multi-AZ la amortigüe.

8

Relacional (RDS), clave-valor (DynamoDB) y documental (DocumentDB): la elección depende de la forma de los datos, no de la moda.